SEA Self-Learning Topic:
Agile vs DevOps, Requirement Elicitation, and Requirement Management

1. Introduction

Modern software is no longer built and shipped the way it was two decades ago. Two ideas sit at the centre of how contemporary teams work:
Agile, which governs how software is developed, and DevOps, which governs how it is delivered and operated.
Around these two ideas sit a set of supporting practices: project management tools, structured requirement-writing formats, requirement elicitation techniques, and requirement traceability that together ensure a team builds the right product, in the right way, without losing track of what was actually asked for. This report examines these topics in sequence, starting from Agile and DevOps fundamentals and moving toward the tools and techniques used to manage requirements in real industry settings, supported by a case study of Netflix's DevOps transformation.

2. Agile vs DevOps

Agile and DevOps are often used interchangeably by beginners, but they address different parts of the software lifecycle. Agile is a development methodology, it defines how a team plans, builds, and iterates on software in short cycles called sprints (typically two to four weeks), with continuous feedback from stakeholders. DevOps, on the other hand, is a culture and set of practices that unifies Development and Operations teams so that code can be built, tested, released, and monitored in a fast, automated, and reliable manner. In short, Agile is concerned with how software is developed, while DevOps is concerned with how it is delivered and operated.

The two are complementary rather than competing. A team can follow Agile ceremonies (sprint planning, daily stand-ups, retrospectives) for development while simultaneously using DevOps practices such as Continuous Integration/Continuous Deployment (CI/CD), automated testing, and infrastructure monitoring to release that work reliably. Most modern software organisations practice both together.

2.1 Comparison Table

Aspect	Agile	DevOps
Primary focus	Software development process	Software delivery, deployment and operations
Core goal	Rapid, iterative development with adaptability to changing requirements	Fast, reliable, and continuous delivery of working software
Origin	Agile Manifesto, 2001	Evolved from Agile principles around 2007–2009
Team structure	Cross-functional dev + product/business team	Development, QA, and Operations teams working as one unit
Key practices	Sprints, backlogs, stand-ups, retrospectives	CI/CD pipelines, automation, containerisation, monitoring
Feedback source	Customer / stakeholder feedback each sprint	Feedback from production systems and monitoring tools
Common tools	JIRA, Trello, Asana	Jenkins, Docker, Kubernetes, GitHub Actions, AWS
Failure handling	Adjust scope/plan in the next sprint	Automated rollback, quick redeployment

As defined by AWS, DevOps combines cultural philosophies, practices, and tools to increase an organisation's ability to deliver applications and services at high velocity, evolving and improving products faster than organisations using traditional software development processes. This definition highlights that DevOps is not merely a toolchain but a cultural shift that removes the traditional wall between developers (who write code) and operations staff (who deploy and maintain it).

3. Case Study: Netflix's DevOps and Agile Transformation

A well-documented example of Agile and DevOps working together at scale is Netflix. In the late 2000s, Netflix was transitioning from shipping physical DVDs to streaming video, and its monolithic data-centre-based architecture repeatedly struggled to handle rapid growth in user traffic. A significant database corruption incident in 2008, which halted DVD shipments for several days, pushed the company to rethink its entire engineering approach.

3.1 What Netflix Changed

- Migrated from a single monolithic application to hundreds of independent microservices, each owned by a small, autonomous Agile team responsible for its own development and deployment.
- Moved its entire infrastructure to Amazon Web Services (AWS), adopting cloud elasticity instead of managing physical servers.
- Built and open-sourced a suite of DevOps tools, including Spinnaker (continuous delivery), Chaos Monkey (which randomly disables production instances to test system resilience), and Atlas (monitoring).
- Adopted a "you build it, you run it" culture, meaning the same engineers who wrote a microservice were also responsible for deploying it and keeping it running in production, the essence of DevOps.
- Used Agile sprints within each microservice team, allowing hundreds of independent teams to release updates several times a day without waiting for a single, company-wide release cycle.

3.2 Outcome

This shift allowed Netflix to deploy code thousands of times per day across its services while maintaining high availability, even though individual components fail constantly in a system of that scale. Chaos Monkey deliberately causes failures in production to ensure that the overall system is resilient, a practice that would be unthinkable without mature CI/CD pipelines and automated monitoring. The Netflix case demonstrates the practical difference explained in Section 2: Agile allowed small teams to iterate quickly on features, while DevOps practices ensured that thousands of daily deployments could happen safely and reliably.

> *Takeaway: Agile made Netflix's teams fast at building; DevOps made Netflix fast and safe at shipping. Neither alone would have been sufficient at Netflix's scale.*

4. Project Management Tools: JIRA vs Asana

Agile and DevOps practices are supported day-to-day by project management software that tracks tasks, sprints, and bugs. Two widely used tools are JIRA and Asana, and they are built for different purposes.

JIRA, developed by Atlassian, is purpose-built for software teams practising Scrum or Kanban. It provides native support for backlogs, sprints, epics, user stories, and bug tracking, and integrates directly with source-code repositories such as Bitbucket and GitHub so that a single commit can be linked to the ticket it resolves. Asana, in contrast, is a general-purpose work-management tool aimed at broader task and

project tracking across any team, marketing, design, operations without built-in concepts such as story points or sprint burndown charts.

4.1 Comparison Table

Feature	JIRA	Asana
Best suited for	Software development teams using Scrum/Kanban	General project & task management across any team
Sprint & backlog support	Native, built-in	Not built-in (possible via workarounds)
Bug tracking	Native issue types (Bug, Story, Task, Epic)	Limited; usually simulated with tags/tasks
Dev tool integration	Deep integration with Bitbucket, GitHub, Confluence	Limited developer-focused integration
Learning curve	Steeper, more configuration options	Simple, intuitive UI
Typical users	Engineering/product teams	Marketing, HR, cross-functional business teams

Example: In a company building a mobile banking app, the engineering team would use JIRA to track a story such as "Implement biometric login," broken into sub-tasks and linked bugs, under an epic titled "Account Security." At the same time, the marketing team promoting the app's launch would use Asana to track tasks like "design launch banner" or "schedule press release," since these tasks involve deadlines and approvals rather than code and sprints.

5. Writing Effective User Stories and Acceptance Criteria

In Agile development, detailed requirement specifications are often replaced with user stories, short, plain-language descriptions of a feature written from the end user's perspective. The standard template is:

As a [type of user], I want [some goal], so that [reason / benefit].

Example: "As a registered student, I want to reset my forgotten password, so that I can regain access to my account."

A user story alone does not define when a feature is complete. Acceptance criteria are the specific, testable conditions that must be satisfied for the story to be considered "done." They give both developers and testers a shared, unambiguous definition of completion.

5.1 Example: User Story with Acceptance Criteria

Element	Content
User Story	As a registered student, I want to reset my forgotten password, so that I can regain access to my account.
Acceptance Criterion 1	User can request a reset link by entering their registered email address.
Acceptance Criterion 2	The reset link expires 24 hours after it is generated.
Acceptance Criterion 3	New password must be at least 8 characters and contain at least one numeral.
Acceptance Criterion 4	A confirmation email is sent once the password is successfully changed.
Acceptance Criterion 5	The old password becomes invalid immediately after the reset is completed.

A widely used format for writing acceptance criteria is Given–When–Then, borrowed from Behaviour-Driven Development (BDD):

> *Given the user is on the login page, When they select "Forgot Password" and enter a valid registered email, Then a password-reset link is sent to that email address.*

6. Advanced Requirement Elicitation Techniques

Requirement elicitation is the process of discovering what stakeholders actually need from a system. Basic techniques such as interviews and questionnaires are common, but they rely on stakeholders being able to accurately articulate their own needs, which is frequently not the case, since people are often unaware of the small inefficiencies in their own routine work. Advanced elicitation techniques address this limitation.

6.1 Interviews

A one-on-one conversation with a stakeholder, which may be structured (a fixed set of questions) or unstructured (a free-flowing discussion). Interviews are effective for gathering detailed, nuanced information but do not scale well when there are many stakeholders with differing views.

6.2 Ethnography (Observation-Based Elicitation)

Rather than asking users what they need, the analyst directly observes them performing their actual work in their real environment over an extended period. For example, if a hospital management system is being built, an analyst might shadow a nurse for an entire shift rather than simply interviewing them. This approach surfaces

requirements that users themselves would never think to mention, such as a manual, repetitive step performed several times a day that could easily be automated, precisely because such habits have become invisible to the people who perform them. Ethnography is more time- and resource-intensive than interviews but tends to produce far more accurate and complete requirements.

6.3 Other Advanced Techniques

- Brainstorming / Focus Groups — Multiple stakeholders are brought together to generate and refine ideas collectively, surfacing requirements through group discussion.
- Prototyping — A rough, working mock-up of the system is shown to users; their reactions and corrections reveal requirements that would not surface through discussion alone.
- Joint Application Development (JAD) Sessions — Structured workshops where developers and stakeholders jointly define requirements in real time, reducing the delay caused by iterative email exchanges.
- Document Analysis — Reviewing existing manuals, forms, and legacy system documentation to understand the current process before defining the new one.

6.4 Comparison Table

Technique	Best Used When	Limitation
Interviews	Detailed input needed from a small set of key stakeholders	Time-consuming at scale; relies on user's self-awareness
Ethnography	Users cannot clearly articulate their workflow	Resource- and time-intensive
Brainstorming	Early-stage idea generation with multiple stakeholders	Can produce unfocused or conflicting ideas
Prototyping	Requirements are unclear or abstract to stakeholders	Risk of users focusing on UI rather than functionality
JAD Sessions	Fast, consensus-driven requirement definition needed	Requires scheduling all key stakeholders together
Document Analysis	A legacy or existing system already exists	Documentation may be outdated or incomplete

7. Requirement Traceability Matrix (RTM)

A Requirement Traceability Matrix is a document that maps each requirement to its origin (the source it came from) and to the artefacts that fulfil it — design documents, implementation, and test cases. Its purpose is to ensure that no requirement is lost, forgotten, or left untested as a project moves from initial client conversations to final delivery.

7.1 Example RTM

Req ID	Requirement Description	Source	Design Doc Ref	Test Case ID	Status
REQ-01	User can reset password via email	Client meeting, 3 Feb	DD-04	TC-12	Completed
REQ-02	Admin can export reports as PDF	Stakeholder email	DD-09	TC-27	In Progress
REQ-03	System supports two-factor authentication	Security review	DD-11	TC-31	Not Started

7.2 Why the RTM Matters

- Ensures every requirement is actually implemented — nothing is dropped during development.
- Ensures every requirement is tested against a specific, traceable test case, rather than QA testing arbitrarily.
- Supports impact analysis — if a requirement changes, the RTM shows exactly which design elements, code modules, and test cases are affected.
- Essential for audits and regulatory compliance in industries such as banking, healthcare, and aviation, where an organisation must formally prove that every requirement was addressed.

Traceability can run in two directions: forward traceability links a requirement to its corresponding test case, while backward traceability links a test case back to the original requirement, useful for checking whether a test case is actually justified by a real requirement, or is redundant.

8. Tools for Requirement Management

For small projects, requirements are often tracked informally in spreadsheets or lightweight tools such as JIRA. However, in large-scale or safety-critical industries like aerospace, automotive, defence, medical devices, formal Requirement Management

(RM) tools are used to maintain rigorous, auditable traceability across thousands of requirements.

8.1 IBM DOORS

IBM DOORS (Dynamic Object-Oriented Requirements System) is one of the most widely used requirement management tools in safety-critical industries. It allows requirements to be stored in a structured, hierarchical database, linked bidirectionally to design elements and test cases (effectively automating the RTM described in Section 7), and tracked for changes over the life of a project. Because a single missed requirement in an aircraft or medical device can have severe consequences, DOORS enforces the kind of rigorous, auditable traceability that spreadsheets cannot reliably provide at scale.

8.2 Other Requirement Management Tools

Tool	Typical Use Case
IBM DOORS	Large-scale, safety-critical projects (aerospace, automotive, defence)
Jama Connect	Cloud-based requirement management, medical devices and automotive
Polarion ALM	Application lifecycle management with strong requirement traceability
Confluence (with JIRA)	Wiki-style requirement documentation linked to JIRA tickets
JIRA (with plugins)	Lightweight requirement tracking for small-to-medium software teams

9. Conclusion

Agile and DevOps together form the backbone of how modern software teams build and deliver products, as demonstrated concretely by Netflix's transformation into an organisation capable of thousands of daily deployments. Tools like JIRA and Asana operationalise these processes on a day-to-day basis, while user stories and acceptance criteria translate stakeholder needs into implementable, testable units of work. Requirement elicitation techniques, particularly advanced approaches such as ethnography, ensure that these user stories are grounded in the user's real needs rather than assumptions. Finally, the Requirement Traceability Matrix and formal tools such as IBM DOORS ensure that, no matter how large or complex a project becomes, every requirement can be traced from its origin to its final, tested implementation. Together, these practices form a coherent pipeline, from understanding what users need, to building it iteratively, to delivering and tracing it reliably.